

Ödev Raporu: SOLID Prensipleri İhlal Analizi ve Clean Code Refactoring

Öğrenci: Aygen Konu: Dart Kodunda SOLID İhlalleri Analizi & Çözümü Teslim Sürümü: v1.1.0 (Release)

Hocam Merhaba,

Ödev kapsamında tarafımıza iletilen sipariş yönetim sistemine ait Dart kodunu satır satır inceledim. Kod içinde kasıtlı olarak bırakılan mimari problemleri araştırdım. İstenen "en az 3 ihlal" kuralının da ötesine geçerek, konuyu tam kavradığımı göstermek adına **5 temel SOLID prensibinin tamamının nasıl ihlal edildiğini** tespit ettim. Tespit ettiğim tüm ihlalleri, kod bloklarını ve sebep oldukları hataları aşağıda sundum. Ardından tüm bu kusurları Clean Code ilkeleriyle ortadan kaldıran refactoring çalışmamı tamamlayıp GitHub depoma v1.1.0 Release etiketiyle yükledim.

1. KODDA TESPİT EDİLEN SOLID İHLALLERİ (DETAYLI ANALİZ)

1. Liskov Substitution Principle (LSP) İhlali

KRİTİK HATA • RUNTIME CRASH

HATALI KOD KESİTİ:

```
class DijitalUrun extends Urun {
  DijitalUrun(String id, String ad, double fiyat, int stok) : super(id, ad, fiyat, stok, "DİJİTAL");

  @override
  double kargoUcretiHesapla() {
    throw Exception("Dijital ürünlerde kargo hesaplanamaz!");
  }
}
// siparisTamamla() fonksiyonunda çağrıldığı an:
toplam += sepet[i].kargoUcretiHesapla();
```

NEDEN İHLAL?:

LSP kuralına göre alt sınıflar (DijitalUrun), üst sınıfın (Urun) yerine geçtiğinde sistemin temel davranışını bozmamalı ve beklenmeyen istisnalar fırlatmamalıdır. Ancak burada DijitalUrun bir Urun olarak sepete eklendiğinde ve sipariş tamamlanmak istendiğinde doğrudan Exception fırlatıyor ve tüm uygulamayı çalışma zamanında (runtime) çökertiyor!

Nasıl Çözdüm?: Kargo hesabı bütün ürünlerin ortak özelliği olamaz. Kargo özelliğini IKargolanabilir arayüzüne taşıdım. Yalnızca FizikselUrun bu arayüzü uygularken, DijitalUrun kargo hesabı yapmaya zorlanmadı. Böylece sepetteki dijital ürünler hiçbir hata vermeden sorunsuz işlenmektedir.

2. Single Responsibility Principle (SRP) İhlali

GOD CLASS • ÇOK FAZLA SORUMLULUK

HATALI KOD KESİTİ:

```
class SiparisYoneticisi implements ISiparisIslemleri {
  // Stok kontrolü yapıyor,
  // Kargo ücretini ve sepet toplamını hesaplıyor,
  // Kupon indirimlerini uyguluyor (INDIRIM10, YAZ20...),
  // KDV hesaplıyor (%20),
  // Ödeme yapıyor (kredi kartı, havale, kapıda ödeme, kripto),
  // Veritabanına SQL yazıyor (INSERT INTO siparisler...),
  // Fatura kesiyor,
  // Hem SMS hem E-posta gönderiyor,
  // Kargo fişi bastırıyor!
}
```

NEDEN İHLAL?:

SRP kuralına göre bir sınıfın değişmesi için yalnızca **tek bir nedeni** olmalıdır. SiparisYoneticisi sınıfı ise adeta "Tanrı Sınıfı" (God Class) haline gelmiştir. Vergi oranı değişse, kupon kuralı değişse, SMS sağlayıcısı değişse veya ödeme yöntemi güncellense hep bu sınıfa dokunmak zorunda kalıyoruz. Bu durum sınıfı aşırı kırılgan ve test edilemez hale getiriyor.

Nasıl Çözdüm?: Sorumlulukları ayrıştırdım: Fatura için IFaturaServisi, bildirimler için IBildirimServisi, kuponlar için KuponServisi, veritabanı için ISiparisRepositoryy oluşturdum. SiparisYoneticisi yalnızca bu süreçlerin koordinatörü haline geldi.

3. Open/Closed Principle (OCP) İhlali

GELİŞİME KAPALI • İF-ELSE BAĞIMLILIĞI

HATALI KOD KESİTİ:

```
@override
void odemeYap(String tip, double tutar) {
    if (tip == "KREDI_KARTI") { ... }
    else if (tip == "HAVALE") { ... }
    else if (tip == "KAPIDA_ODEME") { ... }
    else if (tip == "CRYPTO") { ... }
}
// Kupon kontrolü:
if (kuponKodu == "INDIRIM10") { ... }
else if (kuponKodu == "YAZ20") { ... }
else if (kuponKodu == "SEPETTE50") { ... }
```

NEDEN İHLAL?:

OCP kuralına göre sınıflar *gelişime açık* (open for extension), ancak *değişime kapalı* (closed for modification) olmalıdır. Burada yeni bir ödeme seçeneği (örneğin Apple Pay veya Papara) ya da yeni bir indirim kuponu eklemek istediğimizde, mevcut çalışan sınıfın içine girip if-else bloklarını değiştirmek zorundayız. Bu durum çalışan kodu bozma (regression) riski doğurur.

Nasıl Çözdüm?: **Strategy Pattern** uyguladım. IOdemeYontemi arayüzü ile KrediKartiOdeme, HavaleOdeme gibi bağımsız sınıflar yazdım. Yeni bir ödeme yöntemi veya kupon geldiğinde var olan hiçbir sınıfa dokunmadan sadece yeni bir sınıf eklemek yeterli oldu.

4. Dependency Inversion Principle (DIP) İhlali

SIKI BAĞIMLILIK • TIGHTLY COUPLED

HATALI KOD KESİTİ:

```
class SiparisYoneticisi implements ISiparisIslemleri {
    SqliteVeritabani db = SqliteVeritabani();
    SmtMailServisi mailci = SmtMailServisi();
    NetgsmSmsServisi smsci = NetgsmSmsServisi();
    ...
}
```

NEDEN İHLAL?:

DIP kuralına göre yüksek seviyeli modüller, düşük seviyeli modüllere doğrudan somut olarak bağlanmamalı; her ikisi de soyutlamalara (interface) dayanmalıdır. SiparisYoneticisi somut sınıfları doğrudan kendi içinde new ile üretmiştir. Veritabanını SQLite yerine PostgreSQL yapmak veya SMS servisini değiştirmek istediğimizde bu sınıfı baştan aşağı değiştirmek gerekir. Ayrıca birim testi yazarken bu servisleri mocklamak imkansızdır.

Nasıl Çözdüm?: **Dependency Injection (Constructor Injection)** uyguladım. SiparisYoneticisi artık somut sınıfları değil; ISiparisRepository, IBildirimServisi gibi arayüzleri dışarıdan parametre olarak almaktadır.

5. Interface Segregation Principle (ISP) İhlali

ŞİŞMAN ARAYÜZ • FAT INTERFACE

HATALI KOD KESİTİ:

```
abstract class ISiparisIslemleri {
    void siparisKaydet(String orderId, double tutar);
    void odemeYap(String tip, double tutar);
    void kargoGonder(String orderId, String adres);
    void mailGonder(String email, String mesaj);
    void smsGonder(String tel, String mesaj);
    void faturaYazdir(String orderId);
}
```

NEDEN İHLAL?:

ISP kuralına göre istemciler kullanmadıkları metotlara zorlanmamalıdır. ISiparisIslemleri arayüzü; sipariş, kargo, ödeme, fatura ve bildirim gibi birbiriyle alakasız tüm görevleri tek bir devasa arayüze sıkıştırmıştır. Sadece dijital sipariş işleyen bir sınıf yazmak istesek, hiç kullanmayacağı kargoGonder metodunu mecburen implemente etmek veya hata fırlatmak zorunda kalacaktır.

Nasıl Çözdüm?: Arayüzleri amaca yönelik küçük parçalara böldüm: ISiparisRepository, IKargoServisi, IBildirimServisi, IFaturaServisi.

2. SOLID İHLALLERİ VE CLEAN CODE ÇÖZÜM ÖZETİ

Prensip	Eski Kodun Problemi	Clean Code Çözümümüz
LSP	Dijital ürün kargo metodunda Exception fırlatıp sistemi çökertiyor.	IKargolanabilir arayüzü ayrıldı, dijital ürün hata fırlatmaktan kurtuldu.
SRP	SiparisYoneticisi sınıfı tüm işleri (DB, SMS, Fatura, Ödeme) tek başına yapıyor.	Her iş kendi servis ve repository sınıfına dağıtıldı.
OCF	Ödeme ve kuponlar iç içe if-else bloklarıyla kodlanmış, genişletilemiyor.	Strategy Pattern (IOdemeYontemi, IKuponStratejisi) uygulandı.
DIP	Somut sınıflar (Sqlite, Smtip, Netgsm) içeride doğrudan new ile oluşturuluyor.	Constructor Dependency Injection ile arayüzler dışarıdan enjekte edildi.
ISP	ISiparisIslemleri tüm metotları tek çatı altında toplayıp şişirilmiş.	Odaklı, küçük ve ayrılmış arayüzler (Segregated Interfaces) oluşturuldu.

3. YENİ MİMARİ YAPISI (ARCHITECTURE HIGHLIGHTS)

- Modüler Sınıf Yapısı:** Ürünler (FizikselUrun, DijitalUrun), Ödeme Stratejileri (KrediKarti, Havale, KapidaOdeme, Kripto), Bildirim Servisleri (Mail, SMS), Fatura ve Kargo servisleri birbirinden tamamen izole edildi.
- Güvenli Sepet ve Sipariş Akışı:** Hem dijital e-kitap hem kablosuz fare aynı sepete eklendiğinde dijital ürün için kargo hesaplanmaz, fiziksel ürün için hesaplanır; hiçbir Exception fırlatılmaz.
- Kolay Test Edilebilirlik (Testability):** Bağımlılıklar dışarıdan enjekte edildiği için istenilen mock servislerle %100 birim testi (Unit Test) yazılabilir hale geldi.

4. Proje Deposu & Teslim Bağlantıları (LMS Bilgisi)

Refactor edilmiş çalışan Dart kaynak kodları depoya eklenmiş, test edilmiş ve GitHub üzerinde v1.1.0 etiketiyle resmi Release olarak yayınlanmıştır.

GitHub Depo Bağlantısı: <https://github.com/Aygen5/softito-mobile-online>

GitHub v1.1.0 Release Linki: <https://github.com/Aygen5/softito-mobile-online/releases/tag/v1.1.0>

Refactor Edilmiş Kaynak Kod: lib/solid_refactored_order_system.dart

PDF Belgesi (Depo İçi): [SOLID Odev Raporu.pdf](SOLID_Odev_Raporu.pdf)